

EXERCISE

Position: Intern Developer

Duration: 2 weeks from the date of test confirmation

To address the following problem, the team plans to introduce a **message queue** to enable **asynchronous processing and decouple services**.

Of the following solutions, which is the most appropriate? Prove that your choice is correct based on your knowledge, experience, and experiment.

- *RabbitMQ*
- *Apache ActiveMQ*
- *Apache Kafka*

► CONTEXT

The company is developing a large-scale **e-commerce platform** serving the Vietnamese market. The system currently has approximately **500,000 registered users**, **50,000 daily active users**, and processes around **3,000–5,000 orders per day**, with an expected growth of **3–5 times** within the next **12–18 months**. The backend is built using a **microservices architecture** and deployed on Docker and Kubernetes.

System architecture and services:

- API Gateway exposes REST APIs and handles authentication and rate limiting.
- Order Service receives order creation requests, persists order data to the database, and must return responses quickly without being blocked by downstream services.
- Payment Service processes payments, may be slow or fail intermittently, and requires a retry capability.
- Inventory Service updates product stock levels and is sensitive to race conditions under high concurrency.
- Notification Service sends emails and push notifications and does not require strict real-time delivery.
- Analytics Service collects business events such as order creation, payment success or failure, and inventory updates for reporting and BI, and must not impact the critical user flow.

Currently, all services communicate via **synchronous HTTP REST calls**, which has caused reliability and performance issues as the system scales.

► PROBLEMS

Observed problems in the current design:

- Order creation requests may time out when the Payment Service is slow or unavailable.
- Sending notifications increases the response time of the Order Service.
- Analytics processing degrades overall system performance during traffic spikes.
- Services are tightly coupled, making the system harder to scale and maintain.
- Retry logic is complex and duplicated across multiple services.